

Lab overview

This lab guides me through configuring a web-based employee directory application by integrating Amazon S3 for photo storage and DynamoDB for employee data. I'll create and configure an S3 bucket to store employee images, set up appropriate bucket policies for access, and upload employee photos. Then I'll create a DynamoDB table to store employee information, and learn how to manage employee records through both the web interface and AWS Console. By the end of the lab, I'll have a fully functional employee directory that combines cloud storage and database services to manage employee information and images.

Objectives

After completing this lab, I will be able to:

- Create an Amazon Simple Storage Service (Amazon S3) Bucket
- Create an S3 bucket policy
- Modify an Application to Use an S3 Bucket
- Upload an Object to an S3 Bucket
- Create an Amazon DynamoDB table
- Test an Application Using an Application Web Interface
- Manage Existing DynamoDB Items Using the AWS Management Console
- Create Items in a DynamoDB Table Using the AWS Management Console

Task 1: Create an Amazon Simple Storage Service (Amazon S3) Bucket

☰ [Amazon S3](#) > [Buckets](#) > employee-photo-bucket-ibc-2545

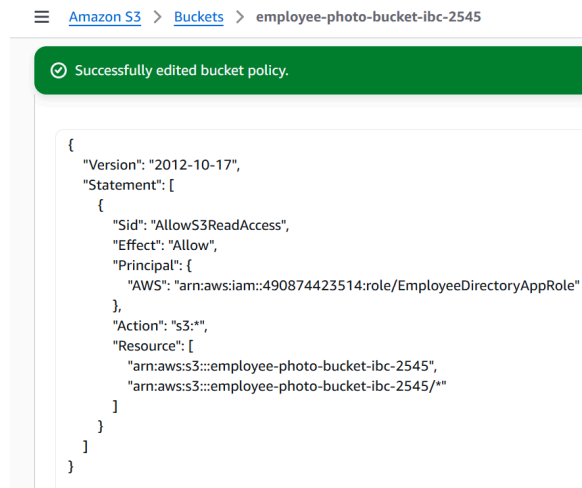
✔ Successfully created bucket "employee-photo-bucket-ibc-2545"

employee-photo-bucket-ibc-2545 [Info](#)

I successfully created an Amazon S3 Bucket for my employee directory web application images.

Task 2: Create an S3 Bucket Policy

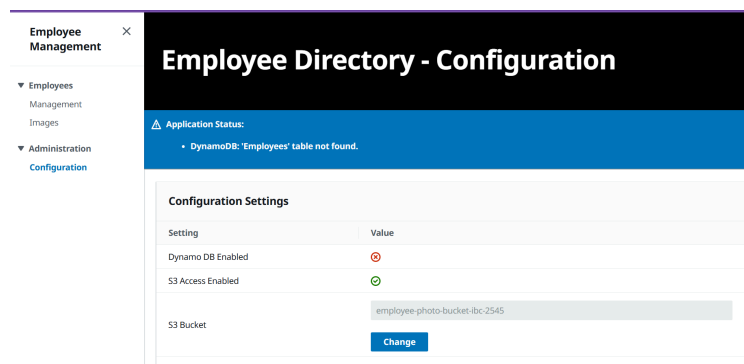
A *bucket policy* contains a set of permissions associated with an S3 bucket. The policy can control access to an entire bucket or to specific directories in a bucket.



I successfully created a bucket policy for my Amazon S3 Bucket.

Task 3: Modify the Application to Use the S3 Bucket

In this task, I'll configure my application to use the bucket as a source for employee images.

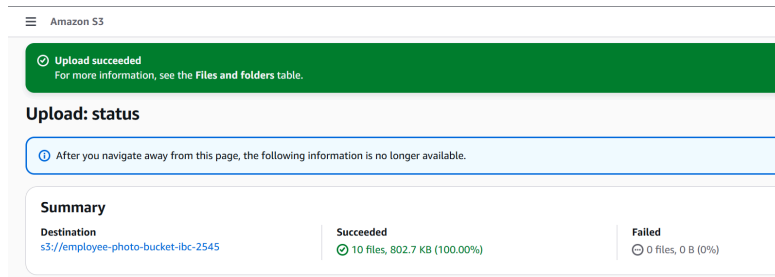


I successfully configured my employee directory web application to use an Amazon S3 Bucket to host my employee images.

Task 4: Upload an Object to an S3 Bucket

I created a bucket and granted permission to it from my EC2 Instance. Next, I'll upload objects (images) to the Amazon S3 Bucket. An *object* can be any kind of file – text, photo, video, .zip, etc. When I add an object to an S3 bucket, I can include custom *metadata* with the object and set *permissions*, providing granular access control.

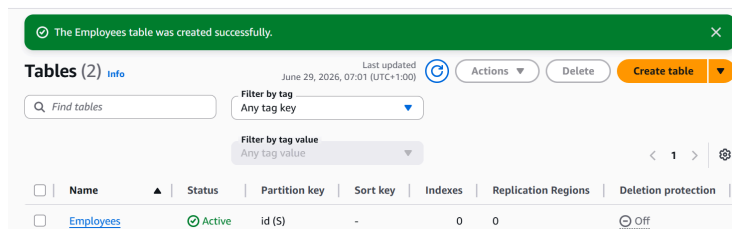
In this task, you upload objects to your S3 bucket.



I successfully uploaded objects to my Amazon S3 Bucket and verified the employee directory application backend configuration.

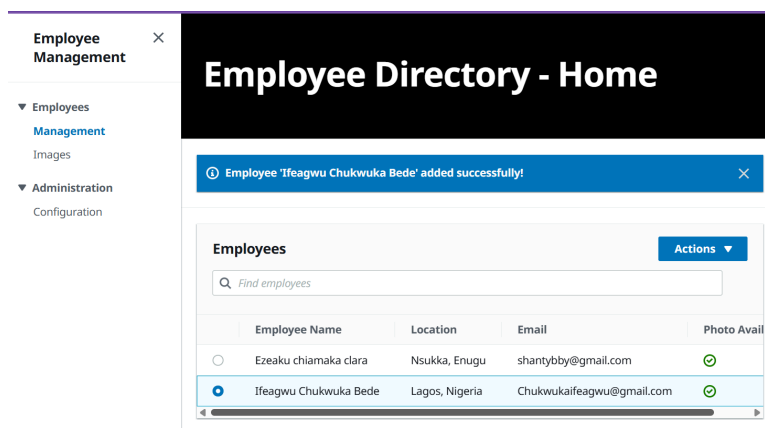
Task 5: Create an Amazon DynamoDB Table

Now that the S3 configuration is complete, the employee directory application is almost ready. But first, I must configure the application to present employee data using Amazon DynamoDB. In this task, I'll create a DynamoDB table to store all of my employee data for the employee directory application.



I successfully created a DynamoDB table.

Task 6: Test the Application Using the Application Web Interface



I successfully tested the employee directory application by adding, editing, and removing items from DynamoDB using the web interface.

Task 7: Manage Existing DynamoDB Items Using the AWS Management Console

In the next task, I'll validate that my new employee data exists in my DynamoDB table and use the AWS Management Console to edit the data in JSON attribute format.

The screenshot displays the 'Employee Directory - Home' application interface on the left and the AWS DynamoDB console on the right. The application shows a table of employees with columns: Employee Name, Location, Email, and Photo Available. The first employee, 'Ifeagwu Chukwuka Bede', is selected. The AWS console shows the 'Employees' table with the following JSON attributes:

```
{
  "id": {
    "S": "dfd5abd5-3a1e-49a8-b9bc-6fe0b48374aa"
  },
  "email": {
    "S": "ifeagwubede@gmail.com"
  },
  "location": {
    "S": "Lagos, Nigeria"
  },
  "name": {
    "S": "Ifeagwu Chukwuka Bede"
  },
  "photo": {
    "S": "employee-1.png"
  }
}
```

I successfully managed existing items in my DynamoDB table and tested the changes in the employee directory application. i.e “email” for the employee “Bede”

Task 8: Create Items in a DynamoDB Table Using the AWS Management Console

In this task, I'll use the AWS Management Console to create DynamoDB items in my table.

The screenshot shows the AWS DynamoDB console with the 'Employees' table. A new item has been added to the table. The JSON attributes for the new item are:

```
{
  "id": {
    "S": "434433ir54"
  },
  "email": {
    "S": "johndoe@gmail.com"
  },
  "location": {
    "S": "new york"
  },
  "name": {
    "S": "john"
  },
  "photo": {
    "S": "employee-5.png"
  }
}
```

The application interface on the right shows the updated table with three employees:

Employee Name	Location	Email	Photo Available
john	new york	johndoe@gmail.com	✓
Ifeagwu Chukwuka Bede	Lagos, Nigeria	ifeagwubede@gmail.com	✓
Ezeaku chiamaka clara	Nsukka, Enugu	shantyby@gmail.com	✓

I successfully added new items to my DYnamoDB table using the AWS Management Console and testing the changes to the employee directory application.

Conclusion

I can now:

- Create an Amazon Simple Storage Service (Amazon S3) Bucket
- Create an S3 bucket policy
- Modify an Application to Use an S3 Bucket
- Upload an Object to an S3 Bucket
- Create an Amazon DynamoDB table
- Test an Application Using an Application Web Interface
- Manage Existing DynamoDB Items Using the AWS Management Console
- Create Items in a DynamoDB Table Using the AWS Management Console